Data driven optimal shared control of Unmanned Aerial Vehicles

Iniya M - CB.SC.U4AIE24219
Poornima P - CB.SC.U4AIE24343
Sowmya A - CB.SC.U4AIE24357
Y. Likitha Reddy - CB.SC.U4AIE24361
CSE (Artificial Intelligence)
Amrita Vishwa Vidyapeetham

August 19, 2026

Abstract

Unmanned Aerial Vehicles (UAVs) are increasingly being used in applications such as disaster response, surveillance, search and rescue, infrastructure inspection, and autonomous navigation. Controlling a UAV is challenging because its dynamics are nonlinear, uncertain, and strongly dependent on the current state and control input. Traditional model-based control approaches require an accurate mathematical model of the UAV, while purely data-driven approaches may not provide an interpretable or control-friendly representation.

This report explains a data-driven UAV control architecture based on the Koopman operator and Extended Dynamic Mode Decomposition (EDMD). EDMD is used to learn a linear representation of nonlinear UAV dynamics in a higher-dimensional observable space. Flight data containing UAV states, control inputs, and subsequent states are collected and transformed using a set of observable functions. The EDMD algorithm then estimates the Koopman operator and obtains matrices describing the evolution of the lifted UAV dynamics.

The learned Koopman model can subsequently be used for prediction and model-based reinforcement learning. An Actor-Critic controller can use these predictions to determine suitable control actions. Finally, a shared-control mechanism combines the human pilot's commands with autonomous commands, allowing the human and AI to cooperate instead of completely replacing one another.

The report presents the complete concept using a real-time disaster-response UAV example, mathematical equations, data-matrix construction, pseudoinverse

computation, prediction, Actor-Critic control, shared control, and integration with PX4, Gazebo, and object detection systems.

Contents

1 Introduction

Unmanned Aerial Vehicles (UAVs), commonly called drones, have become important tools for applications where human access is difficult or dangerous. Disaster response is one such application. After an earthquake, flood, landslide, industrial accident, or building collapse, a UAV can be used to inspect dangerous areas, identify people, detect obstacles, and provide information to rescue teams.

A typical UAV system contains several important components. Sensors measure the state of the vehicle, a flight controller stabilizes and controls the vehicle, a perception system analyzes the environment, and higher-level planning or artificial intelligence determines suitable actions.

A simplified UAV architecture is shown below.

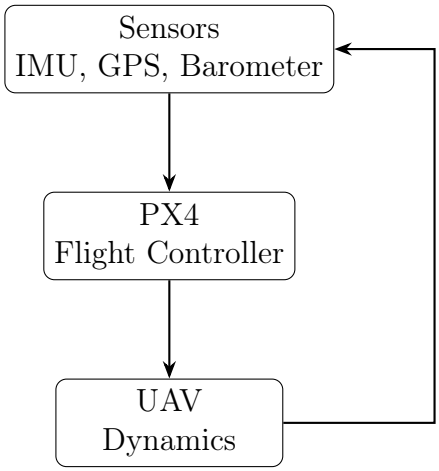

Figure 1: Basic UAV control loop.

The UAV dynamics can generally be written as

$$x_{k+1} = f(x_k, u_k), \tag{1}$$

where x_k represents the UAV state, u_k represents the control input, and $f(\cdot)$ represents the nonlinear UAV dynamics.

For example, an attitude-based state can be represented as

$$x = \begin{bmatrix} \phi & \theta & \psi & p & q & r \end{bmatrix}^T, \tag{2}$$

where ϕ, θ, and ψ represent roll, pitch, and yaw, respectively, while p, q, and r represent angular velocities.

The control vector can be represented as

$$u = \begin{bmatrix} u_\phi & u_\theta & u_\psi \end{bmatrix}^T. \tag{3}$$

The difficulty is that the function f is nonlinear and may be difficult to derive accurately for a real UAV.

This motivates the use of data-driven techniques.

The main idea studied in this report is to learn UAV dynamics directly from flight data using the Koopman operator and Extended Dynamic Mode Decomposition (EDMD).

2 Problem Definition

The main problem is to develop a control architecture in which a UAV can learn its dynamics from measured data and use the learned model to improve autonomous decision-making while still allowing a human pilot to participate in the control loop.

Consider a UAV being manually controlled by a human pilot. The pilot may provide commands such as:

- Move forward
- Move backward
- Move left
- Move right
- Increase altitude
- Decrease altitude
- Rotate

The conventional system is

$$\text{Human} \rightarrow \text{Joystick} \rightarrow \text{PX4} \rightarrow \text{UAV}. \tag{4}$$

The proposed AI-assisted architecture introduces a learned model and an AI controller:

$$\text{Human} + \text{AI Controller} \rightarrow \text{Shared Control} \rightarrow \text{PX4} \rightarrow \text{UAV}. \tag{5}$$

The overall objective is not necessarily to completely remove the human operator. Instead, the AI acts as a co-pilot that can assist the human when the environment becomes difficult or dangerous.

3 Why UAV Dynamics Are Difficult

A UAV is a nonlinear dynamic system. Its motion depends on several interacting variables.

For example, the effect of changing pitch can depend on:

- current pitch,
- current velocity,
- angular velocity,
- thrust,
- aerodynamic effects,
- mass distribution,
- disturbances.

Therefore, a simple linear model

$$x_{k+1} = Ax_k + Bu_k \tag{6}$$

may not accurately represent the complete nonlinear system over a large operating range.

The traditional approach is to derive the UAV equations from physics. However, accurate modeling can become difficult because of uncertainty, disturbances, aerodynamic effects, actuator limitations, and parameter variations.

A data-driven approach instead asks:

> Can the UAV dynamics be learned directly from measured flight data?

Koopman-based methods provide a useful framework for answering this question.

4 Koopman Operator

The Koopman operator provides a way of representing nonlinear dynamics through the evolution of functions of the state.

Consider the nonlinear system

$$x_{k+1} = F(x_k). \tag{7}$$

Instead of directly studying the nonlinear evolution of x, the Koopman approach studies functions of x, called observables.

Let

$$z_k = \Theta(x_k), \tag{8}$$

where Θ is a vector of observable functions.

The Koopman operator $\mathcal{K}$ acts on an observable function $g(x)$ according to

$$(\mathcal{K}g)(x) = g(F(x)). \tag{9}$$

Therefore,

$$\Theta(x_{k+1}) = \mathcal{K}\Theta(x_k). \tag{10}$$

Although the original system may be nonlinear, the Koopman operator is linear in the space of observables.

This is the key idea behind Koopman-based modeling.

5 Observable Functions and Lifting

An observable is simply a mathematical function of the original state.

Suppose a simplified UAV state is

$$x = \begin{bmatrix} \phi \\ \theta \end{bmatrix}. \tag{11}$$

We can define the observable functions

$$\Theta_1(x) = \phi, \tag{12}$$
$$\Theta_2(x) = \theta, \tag{13}$$
$$\Theta_3(x) = \phi^2, \tag{14}$$
$$\Theta_4(x) = \theta^2, \tag{15}$$
$$\Theta_5(x) = \phi\theta. \tag{16}$$

The lifted state is therefore

$$\Theta(x) = \begin{bmatrix} \phi \\ \theta \\ \phi^2 \\ \theta^2 \\ \phi\theta \end{bmatrix}. \tag{17}$$

The original state has two dimensions, while the lifted representation has five dimensions.

This process is called **lifting**.

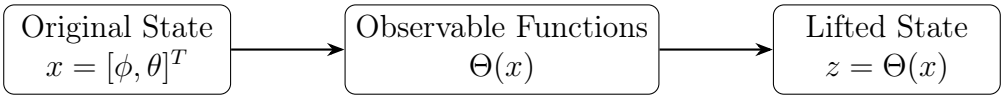

Figure 2: Lifting the UAV state into an observable space.

The choice of observable functions is important because an insufficient dictionary may fail to represent important nonlinear behavior.

6 Controlled Koopman Model

A real UAV is controlled by external inputs. Therefore, the system is more accurately written as

$$x_{k+1} = F(x_k, u_k). \tag{18}$$

The combined state can be represented as

$$\chi_k = \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \tag{19}$$

A vector of basis functions can then be defined as

$$\Theta(\chi_k) = \begin{bmatrix} \Theta_1(\chi_k) \\ \Theta_2(\chi_k) \\ \vdots \\ \Theta_{N_\Theta}(\chi_k) \end{bmatrix}. \tag{20}$$

The Koopman model can be approximated by

$$\Theta(\chi_{k+1}) \approx K^T \Theta(\chi_k), \tag{21}$$

where K is the finite-dimensional approximation of the Koopman operator.

For a controlled linear representation, the model can be written in the form

$$z_{k+1} = Az_k + Bu_k, \tag{22}$$

where

- z_k is the lifted state,
- A represents the natural evolution of the lifted state,
- B represents the effect of control input.

7 Extended Dynamic Mode Decomposition

The Koopman operator is theoretically useful, but its exact infinite-dimensional representation is not directly available for practical UAV control.

Therefore, a finite-dimensional approximation must be learned from data.

Extended Dynamic Mode Decomposition, or EDMD, is a data-driven technique for obtaining such an approximation.

The basic idea is:

$$\boxed{\text{Flight Data} \rightarrow \text{Observable Transformation} \rightarrow \text{EDMD} \rightarrow \text{Koopman Model}} \tag{23}$$

EDMD tries to find an operator that maps the current lifted state to the next lifted state.

The optimization objective can be expressed as

$$E_K = \sum_{j=1}^{N_K} \left\| \Theta(\chi_{j+1}) - K^T \Theta(\chi_j) \right\|^2. \tag{24}$$

The objective is to minimize E_K.

In simple terms, EDMD asks:

> What matrix best explains the transition from the current transformed UAV state to the next transformed UAV state?

8 UAV Data Collection

The first practical step in applying EDMD is collecting UAV flight data.

In a PX4-based implementation, the state can be obtained from the flight controller's state-estimation system. In simulation, the same concept can be implemented using PX4 and Gazebo.

A simplified data flow is

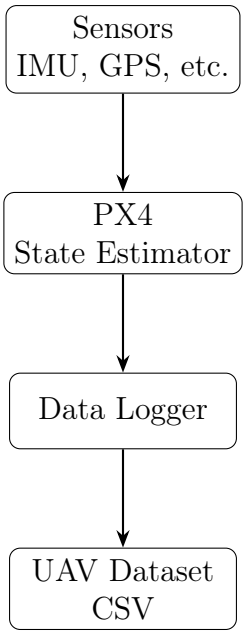

Figure 3: UAV state-data collection pipeline.

A possible state vector is

$$x_k = \begin{bmatrix} \phi_k & \theta_k & \psi_k & p_k & q_k & r_k \end{bmatrix}^T. \tag{25}$$

The control vector can be

$$u_k = \begin{bmatrix} u_{\phi,k} & u_{\theta,k} & u_{\psi,k} \end{bmatrix}^T. \tag{26}$$

Each recorded sample therefore contains

$$(x_k, u_k, x_{k+1}). \tag{27}$$

9 Real-Time UAV Example

Consider a UAV being used to inspect a collapsed building after an earthquake.

The human pilot is controlling the UAV manually.

At time $t = 0$, the UAV has the state

$$x_0 = \begin{bmatrix} 2.0 \\ 1.0 \\ 30.0 \\ 0.10 \\ 0.05 \\ 0.02 \end{bmatrix}. \tag{28}$$

The pilot commands a small left maneuver:

$$u_0 = \begin{bmatrix} 0.2 \\ 0.1 \\ 0 \end{bmatrix}. \tag{29}$$

After 0.1 seconds, PX4 reports

$$x_1 = \begin{bmatrix} 2.4 \\ 1.2 \\ 30.1 \\ 0.15 \\ 0.07 \\ 0.03 \end{bmatrix}. \tag{30}$$

This creates one transition:

$$(x_0, u_0, x_1). \tag{31}$$

The process is repeated thousands of times.

A simplified dataset could look like Table 1.

Table 1: Illustrative UAV flight data.

Time	Roll	Pitch	Yaw	p	q	r	u_ϕ	u_θ	u_ψ
0.0	2.0	1.0	30.0	0.10	0.05	0.02	0.2	0.1	0
0.1	2.4	1.2	30.1	0.15	0.07	0.03	0.2	0.1	0
0.2	3.0	1.4	30.2	0.20	0.08	0.03	0.3	0.1	0
0.3	3.7	1.6	30.3	0.25	0.09	0.04	0.0	0.0	0
0.4	4.0	1.7	30.3	0.10	0.02	0.01	-0.2	0.0	0

The values in this table are illustrative and are used only to demonstrate the EDMD procedure.

10 EDMD Data Matrices

Assume that N samples have been collected.

Let

$$\chi_k = \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \tag{32}$$

After applying the observable dictionary, we obtain

$$\Theta(\chi_k). \tag{33}$$

The current-state matrix is constructed as

$$X_K = \begin{bmatrix} | & | & & | \\ \Theta(\chi_1) & \Theta(\chi_2) & \cdots & \Theta(\chi_{N-1}) \\ | & | & & | \end{bmatrix}. \tag{34}$$

The next-state matrix is

$$Y_K = \begin{bmatrix} | & | & & | \\ \Theta(\chi_2) & \Theta(\chi_3) & \cdots & \Theta(\chi_N) \\ | & | & & | \end{bmatrix}. \tag{35}$$

The corresponding control matrix is represented by

$$U_K = \begin{bmatrix} | & | & & | \\ u_1 & u_2 & \cdots & u_{N-1} \\ | & | & & | \end{bmatrix}. \tag{36}$$

The matrices therefore represent

$$X_K : \text{current lifted states}, \tag{37}$$

$$Y_K : \text{next lifted states}, \tag{38}$$

$$U_K : \text{control inputs}. \tag{39}$$

11 EDMD Matrix Estimation

The controlled EDMD model can be written as

$$Y_K \approx [A \;\; B] \begin{bmatrix} X_K \\ U_K \end{bmatrix}. \tag{40}$$

The matrix $[A \;\; B]$ can be obtained using the Moore–Penrose pseudoinverse.

Thus,

$$\boxed{[A \;\; B] = Y_K \begin{bmatrix} X_K \\ U_K \end{bmatrix}^{\dagger}} \tag{41}$$

where † denotes the Moore–Penrose pseudoinverse.

This is one of the most important equations in the implementation.

The result gives the matrices A and B.

The learned model is then

$$\boxed{z_{k+1} = Az_k + Bu_k} \tag{42}$$

where

$$z_k = \Theta(x_k). \tag{43}$$

The matrix A describes the evolution of the lifted UAV dynamics, while B describes how the control input influences the lifted state.

12 Simple Numerical EDMD Example

To understand the process more easily, consider a one-dimensional UAV roll system.

Let

$$x = \phi. \tag{44}$$

Suppose the observable dictionary is

$$\Theta(x) = \begin{bmatrix} x \\ x^2 \end{bmatrix}. \tag{45}$$

Assume the measured transitions are

Current roll	Next roll
1.0	1.2
2.0	2.5
3.0	3.7

For the first state,

$$\Theta(1) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \tag{46}$$

For the next state,

$$\Theta(1.2) = \begin{bmatrix} 1.2 \\ 1.44 \end{bmatrix}. \tag{47}$$

EDMD attempts to find a matrix K satisfying

$$K^T \begin{bmatrix} 1 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 1.2 \\ 1.44 \end{bmatrix}. \tag{48}$$

However, the actual estimation uses all available samples simultaneously.

This is important because a single sample cannot reliably identify the UAV dynamics.

13 Moore–Penrose Pseudoinverse

The Moore–Penrose pseudoinverse provides a least-squares solution.

Suppose

$$Y = MX. \tag{49}$$

If M is unknown, we can estimate it as

$$M = YX^{\dagger}. \tag{50}$$

For EDMD,

$$Y_K \approx [A \;\; B] \begin{bmatrix} X_K \\ U_K \end{bmatrix}. \tag{51}$$

Therefore,

$$[A \;\; B] = Y_K \begin{bmatrix} X_K \\ U_K \end{bmatrix}^{\dagger}. \tag{52}$$

The pseudoinverse is useful because the collected dataset is generally overdetermined.

For example, suppose there are 50 observables but 10,000 measurements. The algorithm finds the matrix that best fits all these measurements in the least-squares sense.

14 Prediction Using the Learned Model

Once A and B are obtained, the model can be used for prediction.

Given the current lifted state

$$z_k = \Theta(x_k) \tag{53}$$

and a candidate control action u_k, the next lifted state is predicted using

$$\hat{z}_{k+1} = Az_k + Bu_k. \tag{54}$$

The prediction can be repeated multiple steps into the future.
For example,

$$\hat{z}_{k+1} = Az_k + Bu_k, \tag{55}$$
$$\hat{z}_{k+2} = A\hat{z}_{k+1} + Bu_{k+1}, \tag{56}$$
$$\hat{z}_{k+3} = A\hat{z}_{k+2} + Bu_{k+2}. \tag{57}$$

This allows the controller to compare possible future trajectories.

15 Real-Time Obstacle Example

Suppose the UAV is inspecting a damaged building.
The camera detects an obstacle ahead.
The human pilot commands

$$u_h = \text{FORWARD}. \tag{58}$$

The AI considers several candidate actions:

$$u_1 = \text{FORWARD}, \tag{59}$$
$$u_2 = \text{LEFT}, \tag{60}$$
$$u_3 = \text{RIGHT}. \tag{61}$$

For each action, the Koopman model predicts the future state:

$$\hat{z}^{(i)}_{k+1} = Az_k + Bu_i. \tag{62}$$

The controller can then evaluate the predicted states.
For example:

Action	Predicted outcome
Forward	Approaches obstacle
Left	Safer trajectory
Right	Moderate trajectory

The AI may therefore select

$$u^* = \text{LEFT}. \tag{63}$$

This does not mean EDMD itself decides that "LEFT" is the best action. EDMD provides the predictive model. The decision can be made by a controller such as an Actor-Critic policy.

16 EDMD Versus Reinforcement Learning

EDMD and reinforcement learning perform different functions.

16.1 EDMD

EDMD answers:

> How does the UAV behave?

It learns

$$z_{k+1} = Az_k + Bu_k. \tag{64}$$

Therefore,

$$\boxed{\text{EDMD} \rightarrow \text{Dynamics Model}} \tag{65}$$

16.2 Actor-Critic

Actor-Critic answers:

> What action should the UAV take?

The Actor can be represented as

$$u^* = \pi(x), \tag{66}$$

where π is the learned policy.
Therefore,

$$\boxed{\text{Actor-Critic} \rightarrow \text{Control Policy}} \tag{67}$$

The two components can therefore work together.

17 Actor-Critic Control

The Actor-Critic method contains two main components.

17.1 Actor

The Actor generates a control action based on the current state:

$$u_k = \pi_\theta(x_k), \tag{68}$$

where θ represents the neural-network parameters.

For example, the Actor may output

$$u_k = \begin{bmatrix} -0.3 \\ 0.1 \\ 0 \end{bmatrix}. \tag{69}$$

This can correspond to a left maneuver with a small pitch correction.

17.2 Critic

The Critic estimates the value of a state.

A value function can be represented as

$$V(x_k). \tag{70}$$

The Critic evaluates whether the state/action combination is good or bad.

The Actor is then updated to produce better actions.

18 Integration of EDMD with Actor-Critic

The learned Koopman model can be used to provide predicted dynamics for model-based reinforcement learning.

The overall process is

$$x_k \rightarrow \Theta(x_k) \rightarrow Az_k + Bu_k \rightarrow \text{Predicted Future State.} \tag{71}$$

The Actor can evaluate candidate actions using these predictions.

The architecture can be represented as

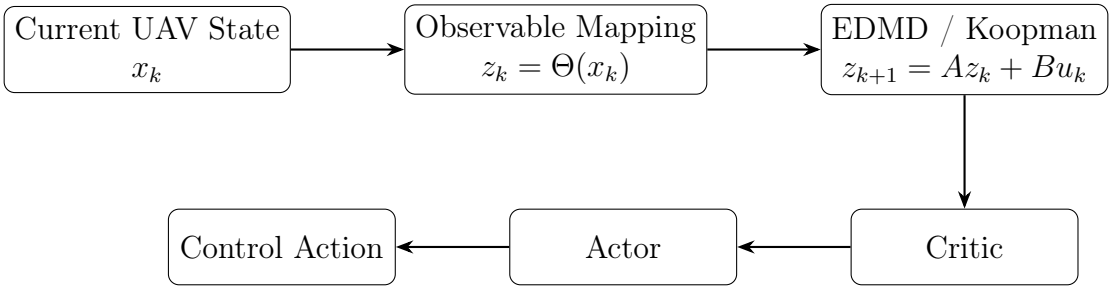

Figure 4: Integration of EDMD and Actor-Critic control.

The paper uses a model-based reinforcement learning framework in which the Koopman model provides a learned description of UAV dynamics and the Actor-Critic method is used to obtain an optimal controller.

19 Human-AI Shared Control

The final component is shared control.

Suppose the human gives

$$u_h = \text{FORWARD} \tag{72}$$

while the autonomous controller gives

$$u_a = \text{LEFT}. \tag{73}$$

A simple shared-control concept can be represented as

$$u_{\text{shared}} = \alpha u_h + (1 - \alpha)u_a, \tag{74}$$

where α represents the human contribution.

However, the paper uses an adaptive shared-control mechanism in which the human command is incorporated into the optimal autonomous command using an adaptive cooperation parameter.

The conceptual form is

$$\hat{U}(i) = \hat{U}(i) + \alpha_i U_h(i), \tag{75}$$

with the adaptive coefficient determining the human contribution.

The goal is to make cooperation smooth rather than abruptly switching authority from human to autonomous control.

20 Complete Real-Time Example

Consider the following disaster-response scenario.

A UAV is flying over a collapsed building.

The human operator wants to inspect the area.

At time t_k:

$$x_k = [\phi_k, \theta_k, \psi_k, p_k, q_k, r_k]^T. \tag{76}$$

PX4 provides the current state.

At the same time, the camera captures an image.

The object detector identifies:

- a person,
- debris,
- a nearby building structure.

The decision engine determines that the UAV should move toward the person while avoiding the debris.

The human pilot gives

$$u_h = \text{FORWARD}. \tag{77}$$

The AI controller considers the same state.

First, the state is lifted:

$$z_k = \Theta(x_k). \tag{78}$$

The learned Koopman model predicts future states:

$$z_{k+1} = Az_k + Bu_k. \tag{79}$$

The Actor-Critic controller decides that moving slightly left is safer:

$$u_a = \text{LEFT}. \tag{80}$$

The shared controller combines the human and autonomous commands.

The final command is sent to PX4.

PX4 controls the UAV.

After the UAV moves, a new state

$$x_{k+1} \tag{81}$$

is measured.

This creates a new transition

$$(x_k, u_k, x_{k+1}), \tag{82}$$

which can be stored for future learning.

The complete loop is therefore

$$\boxed{\begin{aligned}&\text{Camera} \rightarrow \text{Perception}\\ &\qquad\rightarrow \text{Decision Engine}\\ &\qquad\rightarrow \text{AI Controller}\\ &\qquad\rightarrow \text{Shared Control}\\ &\qquad\rightarrow \text{PX4}\\ &\qquad\rightarrow \text{UAV}\\ &\qquad\rightarrow \text{State Measurement}\\ &\qquad\rightarrow \text{EDMD/Koopman.}\end{aligned}} \tag{83}$$

21 Prediction Error and Validation

The EDMD model must be validated before it is used for control.

Let the actual state be

$$x_{\text{actual}} \tag{84}$$

and the predicted state be

$$\hat{x}_{\text{predicted}}. \tag{85}$$

The prediction error is

$$e = x_{\text{actual}} - \hat{x}_{\text{predicted}}. \tag{86}$$

Mean Absolute Error is

$$MAE = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i|. \tag{87}$$

Root Mean Squared Error is

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}. \tag{88}$$

For example, if the actual roll is

$$4.0^\circ$$

and the predicted roll is

$$3.8^\circ,$$

then

$$e_\phi = 4.0 - 3.8 = 0.2^\circ. \tag{89}$$

A low prediction error indicates that the learned model can reproduce the measured UAV dynamics reasonably well.

22 Why EDMD Validation Is Important

The Actor-Critic controller relies on the learned model when model-based prediction is used.

If the EDMD model is inaccurate, then future-state predictions can also be inaccurate.

For example,

Time	0	1	2	3
Actual	1.0	1.2	1.5	1.8
Predicted	1.0	1.21	1.48	1.79

represents a good prediction.

However,

Time	0	1	2	3
Actual	1.0	1.2	1.5	1.8
Predicted	1.0	2.5	4.7	8.9

represents a poor model.

Therefore, EDMD should be validated before using it as the predictive model for the controller.

23 Role of PX4 and Gazebo

PX4 is responsible for low-level flight control and state estimation.

Gazebo can be used to create a simulated environment before deploying the system on a real UAV.

The recommended development pipeline is

$$\text{Gazebo} \rightarrow \text{PX4} \rightarrow \text{State Collection} \rightarrow \text{EDMD}. \tag{90}$$

This allows the initial development to be performed safely in simulation.

A typical workflow is:

1. Start PX4 SITL.
2. Start Gazebo.
3. Make the UAV perform simple maneuvers.
4. Record the UAV state.
5. Record the control input.
6. Generate the dataset.
7. Train the EDMD model.
8. Evaluate prediction accuracy.
9. Develop the Actor-Critic controller.
10. Add shared control.
11. Add perception and object detection.

24 Recommended First EDMD Experiment

It is not recommended to begin with a very large UAV state.

The first experiment can use only roll dynamics.

Define

$$x_k = \begin{bmatrix} \phi_k \\ p_k \end{bmatrix}, \tag{91}$$

where ϕ is roll and p is roll rate.

Let

$$u_k = u_{\phi,k}. \tag{92}$$

The experiment can be

1. Hover.
2. Roll left.
3. Return to center.
4. Roll right.
5. Return to center.

Collect

$$(x_k, u_k, x_{k+1}). \tag{93}$$

Then use a small observable dictionary such as

$$\Theta(x) = \begin{bmatrix} 1 \\ \phi \\ p \\ \phi^2 \\ p^2 \\ \phi p \end{bmatrix}. \tag{94}$$

The resulting EDMD model becomes

$$z_{k+1} = Az_k + Bu_k. \tag{95}$$

Once this works, the state can be expanded.

25 Expanding the Model

After the roll-only experiment, the model can be expanded.

25.1 Experiment 1: Roll

$$x = [\phi, p]^T. \tag{96}$$

25.2 Experiment 2: Roll and Pitch

$$x = [\phi, \theta, p, q]^T. \tag{97}$$

25.3 Experiment 3: Full Attitude

$$x = [\phi, \theta, \psi, p, q, r]^T. \tag{98}$$

25.4 Experiment 4: Position and Velocity

A more complete state can include

$$x = [p_x, p_y, p_z, v_x, v_y, v_z, \phi, \theta, \psi, p, q, r]^T. \tag{99}$$

This staged approach makes it easier to identify problems.

26 Integration with Object Detection

For the complete disaster-response project, the perception module can be based on an object-detection model such as YOLO.

The camera provides an image

$$I_t. \tag{100}$$

The object detector produces detections

$$D_t = \{(\text{class}, \text{confidence}, \text{bounding box})\}. \tag{101}$$

For example:

Object	Confidence
Person	0.92
Obstacle	0.88
Vehicle	0.81

The perception system should not directly control the UAV motors.

Instead,

$$\text{Camera} \rightarrow \text{Object Detection} \rightarrow \text{Decision Engine} \rightarrow \text{AI Controller} \rightarrow \text{PX4}. \tag{102}$$

This provides a clean separation between perception, decision-making, and flight control.

27 Complete System Architecture

The complete proposed architecture can be represented as follows.

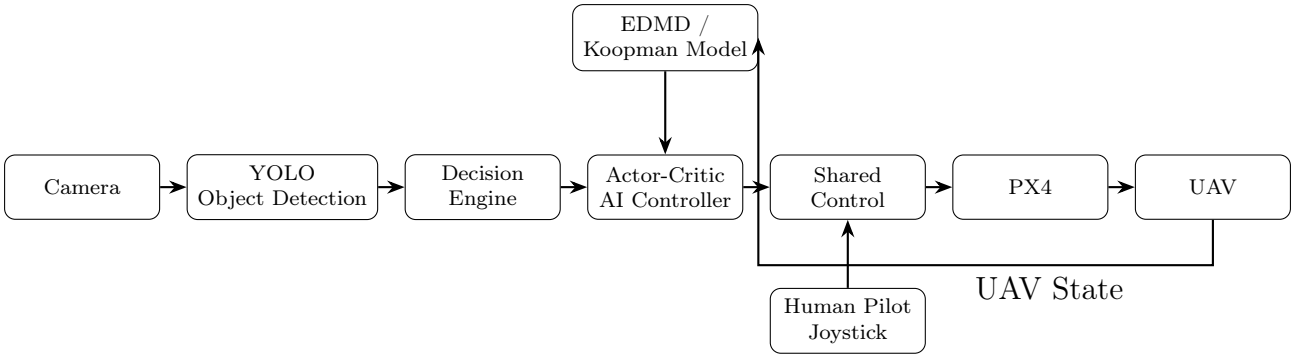

Figure 5: Complete UAV AI co-pilot architecture.

28 Implementation Roadmap

The implementation can be divided into several phases.

28.1 Phase 1: UAV Simulation

Set up

- PX4 SITL,
- Gazebo,
- communication interface,
- basic UAV movement.

28.2 Phase 2: State Collection

Create a state logger that records

$$x_k, u_k, x_{k+1}. \tag{103}$$

Save the data in CSV format.

28.3 Phase 3: Observable Dictionary

Implement functions such as

$$\Theta(x) = [1, x_1, x_2, \ldots, x_1^2, x_2^2, x_1 x_2, \ldots]^T. \tag{104}$$

28.4 Phase 4: EDMD

Construct

$$X_K, \quad Y_K, \quad U_K.$$

Calculate

$$[A \;\; B] = Y_K \begin{bmatrix} X_K \\ U_K \end{bmatrix}^{\dagger}. \tag{105}$$

28.5 Phase 5: Model Validation

Calculate MAE and RMSE.

Plot actual versus predicted trajectories.

28.6 Phase 6: Actor-Critic

Implement the Actor and Critic.

28.7 Phase 7: Shared Control

Combine human and AI commands using the adaptive shared-control mechanism.

28.8 Phase 8: Perception

Integrate YOLO and the decision engine.

29 Advantages of the Approach

The approach has several advantages.

29.1 Data-Driven

The UAV dynamics can be learned from measured data instead of requiring a complete analytical model.

29.2 Nonlinear-System Representation

The Koopman framework provides a way to represent nonlinear dynamics through lifted observables.

29.3 Prediction

The learned model can predict future UAV behavior.

29.4 Model-Based Control

The learned dynamics can be used by a model-based reinforcement-learning controller.

29.5 Human Cooperation

The shared-control mechanism allows the AI to assist rather than completely replace the human operator.

29.6 Adaptability

The model can potentially be updated as additional flight data become available.

30 Disadvantages and Challenges

Despite its advantages, the approach also has limitations.

30.1 Observable Selection

The choice of observable functions is important.

If the dictionary is too small,

$$\text{Model capacity} \downarrow \tag{106}$$

and important nonlinear behavior may not be represented.

If the dictionary becomes extremely large,

$$\text{Computational cost} \uparrow . \tag{107}$$

30.2 Data Quality

EDMD depends strongly on the quality and diversity of collected data.

If the dataset only contains hovering data, the model may perform poorly during aggressive maneuvers.

30.3 Distribution Shift

A model learned in one environment may not accurately represent another environment.

30.4 Noise

UAV measurements contain sensor noise.

Therefore,

$$x_{\text{measured}} = x_{\text{true}} + \eta, \tag{108}$$

where η represents measurement noise.

30.5 Model Error

The finite-dimensional EDMD model is only an approximation of the underlying dynamics.

Therefore,

$$z_{k+1} = Az_k + Bu_k + \epsilon_k, \tag{109}$$

where ϵ_k represents model error.

31 Important Conceptual Differences

It is important to distinguish the major components.

Table 2: Roles of the main components.

Component	Main Function
Koopman Operator	Provides a linear-operator representation of nonlinear dynamics in observable space.
EDMD	Learns a finite-dimensional approximation of the Koopman operator from data.
Actor	Generates control actions.
Critic	Evaluates the quality of states/actions.
Shared Control	Combines human and autonomous control contributions.
PX4	Performs flight-control and state-estimation functions.
YOLO	Detects objects in camera images.
Decision Engine	Converts perception information into high-level decisions.

The easiest way to remember the architecture is

$$\boxed{\begin{array}{rcl} \text{Koopman} & \rightarrow & \text{Representation} \\ \text{EDMD} & \rightarrow & \text{Learning Dynamics} \\ \text{Actor-Critic} & \rightarrow & \text{Decision} \\ \text{Shared Control} & \rightarrow & \text{Cooperation} \\ \text{PX4} & \rightarrow & \text{Flight Control} \\ \text{YOLO} & \rightarrow & \text{Perception} \end{array}} \tag{110}$$

32 Complete Mathematical Summary

The complete mathematical pipeline can be summarized as follows.

The nonlinear UAV system is

$$x_{k+1} = F(x_k, u_k). \tag{111}$$

Define the lifted state

$$z_k = \Theta(x_k). \tag{112}$$

The Koopman-based approximation is

$$z_{k+1} \approx Az_k + Bu_k. \tag{113}$$

From measured data, construct

$$X_K = \begin{bmatrix} z_1 & z_2 & \cdots & z_{N-1} \end{bmatrix}, \tag{114}$$

$$Y_K = \begin{bmatrix} z_2 & z_3 & \cdots & z_N \end{bmatrix}, \tag{115}$$

and

$$U_K = \begin{bmatrix} u_1 & u_2 & \cdots & u_{N-1} \end{bmatrix}. \tag{116}$$

Then estimate

$$\boxed{[A \;\; B] = Y_K \begin{bmatrix} X_K \\ U_K \end{bmatrix}^{\dagger}} \tag{117}$$

and use

$$\boxed{\hat{z}_{k+1} = Az_k + Bu_k} \tag{118}$$

for prediction.
The Actor produces

$$u_a = \pi(x), \tag{119}$$

while the human provides

$$u_h. \tag{120}$$

The shared controller produces a final command

$$u_{\text{shared}} = \mathcal{S}(u_a, u_h), \tag{121}$$

where $\mathcal{S}$ represents the adaptive shared-control mechanism.
The final command is sent to PX4.

33 Final End-to-End Workflow

The complete workflow is

1. Start PX4 and Gazebo.
2. Fly the UAV manually.
3. Collect UAV states.
4. Collect control inputs.
5. Store state transitions.
6. Select observable functions.
7. Lift the UAV state.
8. Construct X_K, Y_K, and U_K.
9. Compute the pseudoinverse.
10. Estimate A and B.
11. Validate the EDMD model.
12. Predict future UAV states.
13. Train the Actor-Critic controller.
14. Generate autonomous commands.

15. Obtain human commands.
16. Combine the commands through shared control.
17. Send the final command to PX4.
18. Measure the new UAV state.
19. Repeat the process.

The complete loop is

$$\boxed{\begin{aligned}&\text{UAV} \rightarrow \text{PX4 State} \rightarrow \text{EDMD} \rightarrow \text{Koopman Prediction}\\ &\qquad \rightarrow \text{Actor-Critic} \rightarrow \text{Shared Control} \rightarrow \text{PX4} \rightarrow \text{UAV}.\end{aligned}} \tag{122}$$

34 Conclusion

This report presented a complete explanation of a Koopman-based UAV shared-control architecture.

The main challenge in UAV control is the nonlinear and uncertain nature of UAV dynamics. Instead of relying entirely on a manually derived mathematical model, flight data can be collected from PX4 and used to learn a data-driven representation.

The Koopman operator provides the theoretical framework for representing nonlinear dynamics through the evolution of observable functions. EDMD provides the practical algorithm for estimating a finite-dimensional approximation of this operator from data.

The main EDMD workflow is

$$\boxed{\text{Collect Data} \rightarrow \text{Choose Observables} \rightarrow \text{Lift Data} \rightarrow \text{Construct Matrices} \rightarrow \text{Pseudoinverse} \rightarrow A, B} \tag{123}$$

The resulting model

$$z_{k+1} = Az_k + Bu_k \tag{124}$$

can predict UAV behavior.

The predicted dynamics can then be integrated with Actor-Critic reinforcement learning. The Actor generates autonomous actions while the Critic evaluates their quality. Finally, a shared-control mechanism combines autonomous and human commands so that the AI acts as a co-pilot rather than simply taking complete control.

For the proposed disaster-response UAV project, this architecture can be further integrated with an object-detection module such as YOLO. The perception system identifies

people and obstacles, the decision engine interprets this information, the AI controller determines appropriate actions, and the shared-control layer coordinates those actions with the human operator.

The recommended implementation strategy is to begin with PX4 and Gazebo, collect a small and well-defined dataset, implement a simple roll-based EDMD model, validate the prediction accuracy, and then progressively add pitch, yaw, position, velocity, Actor-Critic control, shared control, and finally object detection.

Therefore, the fundamental idea can be summarized in one sentence:

EDMD learns how the UAV behaves from flight data, allowing the AI controller to predict the effect of control actions and assist the human pilot.

Key Equations at a Glance

$$x_{k+1} = F(x_k, u_k), \tag{125}$$

$$z_k = \Theta(x_k), \tag{126}$$

$$z_{k+1} \approx Az_k + Bu_k, \tag{127}$$

$$E_K = \sum_{j=1}^{N_K} \left\| \Theta(\chi_{j+1}) - K^T \Theta(\chi_j) \right\|^2, \tag{128}$$

$$[A \;\; B] = Y_K \begin{bmatrix} X_K \\ U_K \end{bmatrix}^{\dagger}, \tag{129}$$

$$\hat{z}_{k+1} = Az_k + Bu_k, \tag{130}$$

$$e = x_{\text{actual}} - x_{\text{predicted}}, \tag{131}$$

$$MAE = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|, \tag{132}$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2}, \tag{133}$$

$$u_a = \pi(x), \tag{134}$$

$$u_{\text{shared}} = \mathcal{S}(u_a, u_h). \tag{135}$$